

JOMO KENYATTA UNIVERSITY
OF
AGRICULTURE AND TECHNOLOGY (JKUAT)
SCHOOL OF ELECTRICAL, ELECTRONIC AND INFORMATION ENGINEERING
DEPARTMENT OF TELECOMMUNICATION AND INFORMATION ENGINEERING

ICS 2403 DISTRIBUTED COMPUTING AND APPLICATIONS

PRACTICAL 1 REPORT

DATE : 6TH NOVEMBER, 2026

Group members

NAME	REG. NO	SIGNATURE
NIKITA KAMAU	**ENE221-0102/2022**	
CHRISPINE ODHIAMBO	**ENE221-0132/2020**	
EUNICE MWILU	**ENE221-0158/2022**	
MARK GAITURI	**ENE221-0232/2019**	
BRIAN BANGE	**ENE221-0135/2021**	
BRIAN SANE	**ENE221-0118/2022**	

1.0 Introduction

The aim of this milestone was to investigate how distributed systems maintain service availability when individual nodes experience failures. In a distributed environment, hardware failures, communication interruptions and service crashes can affect the availability of applications.

For this practical, we used a simulated telecom session-routing service to investigate different fault-tolerance and recovery mechanisms. The main focus was on replication, failure detection, leader election and quorum-based coordination.

The simulation compared different system configurations to determine how effectively they recover from failures and maintain service availability. It also examined the relationship between failure detection timeouts and false failovers.

The practical was conducted using Python, with a fault-injection framework that introduced different types of failures into the system.

2.0 Objectives

The objectives of this milestone were:

1. To understand the importance of fault tolerance in distributed systems.
2. To simulate different types of node and communication failures.
3. To investigate how replication improves service availability.
4. To implement heartbeat-based failure detection.
5. To examine leader election and failover mechanisms.
6. To compare primary-backup replication with majority-quorum replication.
7. To measure system availability, mean time to recovery (MTTR) and split-brain exposure.
8. To determine suitable failure-detection timeout values.

3.0 System Design and Experimental Setup

The experiment was designed around a distributed session routing service representing a telecom application.

Different replication configurations were simulated to evaluate their behaviour under various fault conditions.

3.1 Replication Configurations

Four configurations were investigated.

Configuration	Description
A	Single node with restart
B	Primary-backup replication with 2 nodes
C	Majority-quorum replication with 3 nodes
D	Majority-quorum replication with 5 nodes

Configuration A represented a system without replication. If the node failed, the service had to wait for the node to be repaired and restarted.

Configuration B used a primary node and a backup node. If the primary failed, the backup could take over without waiting for the original node to recover.

Configurations C and D used majority-quorum replication. This meant that a majority of the available replicas had to agree before a leader could be elected and the service could continue operating.

3.2 Fault Model

Several types of failures were introduced during the simulation.

Fault Type	Description	Recovery Mechanism
Crash failure	A node completely stops operating.	Detection, election and recovery
Service failure	The process remains alive but cannot provide services.	Health checks and failover
Communication failure	A node becomes unreachable due to network partitioning.	Detection and leader election
Omission failure	Heartbeat messages are dropped.	Failure suspicion and possible failover
Byzantine failure	A node behaves incorrectly or sends invalid information.	Analysed theoretically, not simulated

The simulation used a fault distribution of 40% crash failures, 20% service failures, 25% communication failures and 15% omission failures.

Byzantine failures were discussed separately because they require more advanced fault-tolerance mechanisms.

3.3 Failure Detection

The system used heartbeat messages to monitor the availability of nodes.

Each node was expected to respond to periodic heartbeat requests. When a node failed to respond for a specified number of consecutive heartbeats, it was considered suspicious.

The parameter hb_k represented the number of missed heartbeats required before a node was suspected.

Different detection thresholds were tested to determine how they affected recovery time and false failovers.

4.0 Experimental Methodology

The experiment involved running the different replication configurations under controlled fault conditions.

The following parameters were used:

Parameter	Value
Simulation tick	1 second
Simulation duration	40,000 seconds per run
Baseline MTBF	4,000 seconds
Failure detection timeout	3 heartbeats
Election time	2 seconds
Catch-up time	5 seconds
Single-node restart time	10 seconds
Number of random seeds	8
Fault-rate values	1,000–8,000 seconds MTBF

Two types of fault injection were used.

Stochastic fault injection: Failures were introduced randomly according to the configured failure rate.

Scripted fault injection: Failures were introduced at specific times to observe how the system responded to a known failure event.

The main performance indicators measured were availability, MTTR, number of outages, false failovers and split-brain exposure.

5.0 Results and Observations

5.1 Baseline Availability Results

The baseline experiment was conducted using an MTBF of 4,000 seconds and a failure detection timeout of three heartbeats.

The following results were obtained:

Configuration	Availability	Downtime (min/year)	MTTR (s)
A: Single node	98.168%	9,630	81.9
B: Primary-backup	99.907%	491	4.0
C: Quorum (3 nodes)	99.867%	696	5.1
D: Quorum (5 nodes)	99.904%	504	3.9

The results indicate that replication significantly improved system availability. The single-node configuration recorded an availability of approximately 98.17%, which was considerably lower than the replicated configurations.

The primary-backup configuration achieved the highest baseline availability at 99.907%. The five-node quorum configuration also performed well, achieving 99.904%. The single-node configuration had a significantly higher recovery time because the service depended on repairing and restarting the failed node.

In contrast, the replicated configurations could transfer service to another available node through leader election.

5.2 Effect of Failure Rates on Availability

The next experiment investigated how increasing the frequency of failures affected the different configurations.

Configuration	MTBF 8000 s	MTBF 4000 s	MTBF 2000 s	MTBF 1000 s
Single node	98.678%	98.168%	96.196%	91.503%
Primary-backup	99.955%	99.907%	99.727%	99.242%
Quorum (3 nodes)	99.934%	99.867%	99.592%	98.495%

From the results, I observed that availability decreased as the failure rate increased. The single-node configuration experienced the greatest reduction in availability. When the MTBF reduced to 1,000 seconds, its availability dropped to approximately 91.50%. The primary-backup configuration maintained better availability under the same conditions, achieving 99.24%.

However, the three-node quorum configuration dropped to 98.50% under heavy fault conditions.

This showed that quorum-based systems do not always provide better availability than primary-backup systems. A quorum requires a majority of nodes to remain available, and frequent failures can make it difficult to maintain that majority.

5.3 Effect of Failure Detection Timeout

Another experiment investigated how the number of missed heartbeats affected failure detection and recovery.

The results were as follows:

Missed Heartbeats (hb_k)	Detection Time (s)	Availability	MTTR (s)	False Failovers per Run
1	1	97.863%	2.1	399.1
2	2	99.857%	3.6	5.6
3	3	99.867%	5.1	1.0
5	5	99.831%	7.1	0.5

The results showed that reducing the failure detection timeout improved recovery speed. However, setting the timeout too low caused the system to incorrectly suspect healthy nodes. When the timeout was set to one heartbeat, the system recorded approximately 399 false failovers per run. This resulted from the background heartbeat loss rate of 1%. A timeout of three heartbeats provided a better balance between recovery speed and false failure detection. This demonstrated that failure detection must consider normal communication interruptions before declaring a node unavailable.

5.4 Scripted Failure Scenario 1: Leader Crash

In this experiment, the leader node was deliberately crashed at time 100 seconds. The repair time was set to 120 seconds.

The following results were recorded:

Configuration	Number of Outages	Longest Outage (s)
Single node	1	135
Primary-backup	1	4
Quorum (3 nodes)	1	4
Quorum (5 nodes)	1	4

For the three-node quorum configuration, the sequence of events was as follows:

- At 100 seconds, a crash was introduced on node 0.
- The service became unavailable.
- At 102 seconds, node 0 was suspected after three missed heartbeats.
- The leader election process started.
- Node 1 became the new leader.
- At 104 seconds, the service was restored.
- At 220 seconds, the original node completed its repair.

This showed that replication allowed the service to recover without waiting for the failed node to restart. The replicated configurations restored service in approximately four seconds, compared to 135 seconds for the single-node configuration.

5.5 Scripted Failure Scenario 2: Network Partition

The second scenario investigated the effect of a network partition.

The leader was isolated from the other nodes at 100 seconds for a duration of 60 seconds.

Configuration	Outages	Longest Outage (s)	Split-Brain Exposure (s)
Single node	2	70	0
Primary-backup	1	4	58
Quorum (3 nodes)	1	4	0
Quorum (5 nodes)	1	4	0

The primary-backup configuration experienced 58 seconds of split-brain exposure. This happened because the isolated primary continued accepting writes while the backup was already providing service. The quorum configurations prevented this behaviour because the minority side could not obtain the required majority to continue accepting writes.

This demonstrated the importance of quorum-based coordination in preventing conflicting operations during network partitions.

6.0 Analysis and Discussion

From the experiments, I established that replication improves service availability by allowing another node to take over when the current leader fails.

The primary-backup configuration performed well because it could continue operating with the backup node after the primary failed. However, it had a weakness during network partitions because both the original primary and backup could accept writes simultaneously. The quorum-based configurations addressed this problem by requiring a majority agreement before serving requests.

However, quorum introduces an availability trade-off. When too many nodes become unavailable, the remaining nodes may not have enough members to form a majority. The experiments also showed that failure detection timeouts must be selected carefully. Very short timeouts cause unnecessary failovers, while longer timeouts increase recovery time.

For the simulated telecom system, a timeout of two to three heartbeats provided a reasonable balance.

7.0 Byzantine Fault Analysis

Byzantine failures were not simulated during this practical, but their implications were analysed. A majority-quorum system with $n = 2f + 1$ replicas can tolerate up to f crash failures.

Therefore:

- A three-node quorum can tolerate one crash failure.
- A five-node quorum can tolerate two crash failures.

However, Byzantine fault tolerance requires at least $3f + 1$ replicas. For example, tolerating one Byzantine node requires at least four replicas.

This is because Byzantine nodes can behave unpredictably or send incorrect information instead of simply stopping. For a telecom application, this could involve a replica sending corrupted routing information. Therefore, crash-fault tolerance alone may not be sufficient for systems that require protection against malicious or arbitrary node behaviour.

8.0 Failure-Driven Engineering Log

Question	Observation
What changed?	A fault-injection framework, heartbeat-based failure detection, leader election and quorum-based serving rules were introduced.
What failed?	Very short heartbeat timeouts caused excessive false failovers. Primary-backup also experienced split-brain exposure during network partitioning.
Why did it fail?	A single lost heartbeat was treated as a failure when the timeout was one heartbeat. The primary-backup configuration did not require a majority before serving.
How was it fixed?	The heartbeat threshold was increased to at least two. The split-brain measurement was also corrected to count only actual coexistence of an old and new leader without quorum fencing.
What alternatives were considered?	Adaptive failure detectors and five-node quorum replication were considered.
What was learned?	Replication improves availability, while quorum improves safety. However, both introduce different trade-offs that must be considered when designing a distributed system.

9.0 Conclusion

In conclusion, this milestone demonstrated the importance of fault tolerance and recovery in distributed systems.

The experiments showed that replicated systems achieved significantly higher availability than a single-node configuration. The primary-backup configuration achieved 99.907% availability, while the three-node and five-node quorum configurations achieved 99.867% and 99.904%, respectively.

The experiments also demonstrated that quorum-based replication prevents split-brain exposure, although it can reduce availability when failures are frequent.

Furthermore, I learned that failure detection requires a balance between quick recovery and avoiding false failovers.

For the telecom resource management system, replication and leader election provide an effective way of maintaining service availability when individual nodes fail.

MILESTONE 8 – DISTRIBUTED SYSTEM MODELS

10.0 Introduction

The aim of this milestone was to investigate different distributed computing models and evaluate how they affect system performance, scalability and reliability.

Distributed systems can be organised in different ways depending on how processing resources are allocated and how workloads are handled.

For this practical, I compared five main distributed system models: host-based, workstation, processor pool, server-based and integrated systems.

The experiment used a simulated telecom workload to determine how each architecture handled incoming requests, particularly when workloads were unevenly distributed across nodes.

The main focus was on response time, resource utilisation, network communication overhead and system behaviour during node failure.

11.0 Objectives

The objectives of this milestone were:

1. To understand different distributed system models.
2. To compare host-based and processor-pool architectures.
3. To investigate the effect of workload migration between nodes.
4. To evaluate the effect of network communication on processing latency.
5. To examine the performance of integrated distributed architectures.
6. To measure system scalability under different workloads.
7. To investigate how different architectures respond to node failures.

8. To recommend a suitable distributed computing model for the telecom application.

12.0 Distributed System Models

The experiment compared the following models.

12.1 Host-Based Model

In the host-based model, each node manages its own processes and sessions.

For the telecom application, this represents a situation where each base station or edge device handles the sessions assigned to it.

One disadvantage of this model is that workloads may not be distributed evenly. If one node receives more requests than others, it may become overloaded while other nodes remain underutilised.

12.2 Workstation Model with Migration

The workstation model allows idle nodes to assist nodes experiencing high workloads.

In this experiment, a process could be migrated when the waiting time in its home queue exceeded five milliseconds.

Migration introduced an additional communication cost of three milliseconds.

This approach helped distribute workloads more evenly without requiring all processing to take place in a central location.

12.3 Processor Pool Model

The processor pool model uses a central collection of processing resources shared by incoming requests.

Instead of permanently assigning workloads to individual nodes, requests are distributed among the available processors.

For the telecom system, this represents a central cloud or core computing environment where multiple sessions share available processing resources.

One advantage is that workloads can be distributed more efficiently across the available processing capacity.

12.4 Server-Based Model

In the server-based model, clients communicate with servers that process their requests.

The experiment used four processors per server and a network round-trip time of two milliseconds.

This model introduces network communication overhead because requests must be sent to the appropriate server before processing.

12.5 Integrated Model

The integrated model combines edge processing with a central processor pool. Latency-critical requests are handled by edge processors, while other requests are handled by the central pool.

Two variations were investigated:

- Integrated model with a 50/50 capacity split.
- Integrated model with capacity matching.

The capacity-matched configuration allocated edge processors in proportion to the latency-critical workload, which was approximately 40% of requests.

13.0 Experimental Methodology

The simulation used Poisson request arrivals and exponential service times.

The main experimental parameters were:

Parameter	Value
Requests per run	60,000
Service rate per processor	100 requests/second
Mean service time	10 ms
Default offered load	0.7 per processor
Load range	0.3–0.85
Latency-critical requests	40%
Pool network RTT	1 ms
Server network RTT	2 ms
Migration cost	3 ms
Migration threshold	5 ms
Number of seeds	5
Node counts	4–64

The simulation also introduced a node failure in which node 0 was unavailable during the middle 20% of the experiment. The performance indicators included mean response time, 95th-percentile latency, success rate, SLA violations and network messages per request.

14.0 Results and Observations

14.1 Scalability Results

The first experiment compared the performance of the different models at different cluster sizes. The following table shows selected results at 4, 16 and 64 nodes.

Nodes	Model	Mean Latency (ms)	P95 Latency (ms)	SLA Violations
4	Host-based	37.2	117.9	25.17%
4	Workstation	15.2	39.4	1.88%
4	Processor pool	14.5	38.2	1.62%
4	Server-based	15.5	39.2	1.79%

4	Integrated capacity-matched	237.0	747.8	60.47%
16	Host-based	34.1	103.5	22.85%
16	Workstation	12.2	32.3	0.85%
16	Processor pool	11.2	31.2	0.75%
16	Server-based	15.5	39.3	1.82%
16	Integrated 50/50	55.7	170.9	35.41%
16	Integrated capacity-matched	12.8	34.2	1.02%
64	Host-based	33.0	98.6	21.91%
64	Workstation	11.9	32.0	0.82%
64	Processor pool	11.0	30.9	0.73%
64	Server-based	15.6	39.6	1.91%
64	Integrated capacity-matched	11.2	31.2	0.75%

The results showed that the processor pool generally achieved the lowest latency. The host-based model performed poorly because each node was responsible for its own workload. When one node received more requests than its capacity could handle, its queue continued growing. The workstation model improved performance because it allowed workloads to move to idle nodes.

The server-based model performed slightly worse than the processor pool due to additional network communication delays. The integrated model also performed differently depending on how processing capacity was divided between the edge and central pool.

14.2 Load Sweep Results

The second experiment investigated how increasing workload affected the different architectures.

The following table shows the 95th-percentile latency and SLA violations at 16 nodes.

Load	Host-Based	Workstation	Processor Pool	Server-Based	Integrated 50/50	Integrated Matched
0.30	42.8 / 3.07%	30.8 / 0.73%	30.9 / 0.73%	32.1 / 0.82%	30.7 / 0.71%	30.7 / 0.71%
0.50	60.3 / 8.31%	31.4 / 0.77%	31.0 / 0.73%	33.4 / 0.95%	31.8 / 0.79%	31.0 / 0.73%
0.70	103.5 / 22.85%	32.3 / 0.85%	31.2 / 0.75%	39.3 / 1.82%	170.9 / 35.41%	34.2 / 1.02%
0.85	420.3 / 47.93%	35.1 / 1.13%	33.4 / 0.95%	59.1 / 8.67%	9100.9 / 72.03%	716.2 / 69.32%

The results showed that increasing the workload had different effects on each architecture. The host-based model experienced a significant increase in latency because heavily loaded nodes could not easily transfer their workloads to other nodes.

The workstation and processor pool models maintained relatively stable latency over most of the load range. The integrated 50/50 configuration experienced severe performance degradation because the central processor pool did not have enough capacity to handle the workload being transferred to it. The capacity-matched integrated model performed better at moderate loads, but also experienced saturation at a load of 0.85.

This demonstrated the importance of allocating processing resources according to actual workload requirements.

14.3 Reliability Under Node Failure

The third experiment investigated how each architecture responded when one node failed.

Node 0 was removed during the middle 20% of the simulation, with the offered load maintained at 0.7 per processor.

The following results were recorded:

Model	Overall Success	Success During Failure	P95 Latency (ms)	SLA Violations
Host-based	98.52%	92.61%	102.0	23.69%
Workstation	100%	100%	32.4	0.85%
Processor pool	100%	100%	31.3	0.75%
Server-based	100%	100%	44.9	3.37%
Integrated 50/50	100%	100%	554.7	54.87%
Integrated capacity-matched	100%	100%	35.2	1.16%

The host-based model was the only architecture that lost requests during the experiment. This happened because sessions were attached to individual nodes, and the failed node could not transfer its existing sessions to another processor. The other models were able to reroute requests to available resources.

However, maintaining a 100% success rate did not necessarily mean that the system maintained good performance. For example, the integrated 50/50 configuration recorded a 95th-percentile latency of 554.7 milliseconds despite achieving 100% success.

This showed that a system can remain operational while still experiencing significant performance degradation.

14.4 Network Communication Cost

The final experiment compared the number of network messages required per request.

Model	Network Messages per Request
Host-based	0.00
Workstation with migration	0.92
Processor pool	2.00
Server-based	2.00
Integrated 50/50	1.41
Integrated capacity-matched	1.46

The host-based model had no additional network messages per request because processing occurred locally. The workstation model required additional communication when workloads were migrated. The processor pool and server-based models required two messages per request. The integrated models had intermediate communication overhead because some requests were handled locally

while others were sent to the central pool. Hardware cost was kept equal across the models by using 16 processors in each configuration.

Therefore, the main differences were in network communication and system performance rather than hardware capacity.

15.0 Analysis and Discussion

From the results, I observed that the processor pool model generally provided the best response times because it allowed workloads to share processing resources.

The host-based model performed poorly under uneven workloads because an overloaded node could not easily use the idle capacity of other nodes.

The workstation model addressed this limitation through process migration. It achieved performance close to the processor pool while requiring fewer network messages per request.

The server-based model introduced additional network latency, which affected its response times compared to the processor pool.

The integrated model demonstrated that combining edge and central processing can be effective, but only when processing capacity is allocated according to workload requirements.

The 50/50 integrated configuration performed poorly because the pool received approximately 60% of the traffic while having only half of the processors.

The capacity-matched configuration performed better because approximately 40% of the processors were allocated to edge processing, matching the proportion of latency-critical requests.

The results therefore showed that architecture design should consider workload distribution and resource requirements rather than simply dividing resources equally.

16.0 Recommendation for the Telecom System

Based on the experimental results, I would recommend using a capacity-matched integrated architecture for the telecom resource management system.

This architecture combines the advantages of edge processing and centralised resource pooling.

Latency-critical telecom functions can be handled closer to the users through edge processors, reducing communication delays.

Other workloads can be processed using a central processor pool, where resources can be shared more efficiently.

Workstation-style migration can also be introduced to allow overloaded edge nodes to transfer excess workloads to available processors.

However, the architecture must be properly provisioned to avoid overloading the central pool.

The experiment showed that the integrated architecture performs well at moderate loads but experiences significant degradation when the system approaches saturation.

For this reason, the edge tier should be maintained below approximately 0.8 utilisation under the tested conditions.

17.0 Failure-Driven Engineering Log

Question	Observation
What changed?	Five distributed computing models and a capacity-matched integrated variant were introduced and compared.
What failed?	The host-based model experienced high latency under uneven workloads. The 50/50 integrated model also experienced severe latency due to insufficient pool capacity.
Why did it fail?	Workloads were not evenly distributed across hosts, and the integrated configuration allocated insufficient resources to the central pool.
How was it fixed?	The workload distribution was adjusted to use a controlled hotspot model. A capacity-matched integrated configuration was also introduced.
What alternatives were considered?	Zipf-based workload distribution, dynamic capacity allocation and stability checks for individual nodes were considered.
What was learned?	Performance depends on how processing capacity is distributed. A distributed architecture must be designed around actual workload requirements rather than assuming that equal resource allocation always produces the best results.

18.0 Conclusion

In conclusion, this milestone demonstrated the differences between various distributed computing models and how they affect system performance, reliability and scalability.

The processor pool model generally achieved the lowest latency because it allowed processing resources to be shared across the system.

The workstation model also performed well because it allowed overloaded nodes to transfer workloads to idle nodes.

The host-based model was more vulnerable to uneven workload distribution and node failures because sessions were attached to individual hosts.

The integrated model demonstrated that edge processing and centralised processing can work together effectively when resources are allocated according to workload requirements.

Based on the findings, the capacity-matched integrated architecture was identified as the most suitable recommendation for the simulated telecom application because it combined low latency, workload sharing and resilience to node failures.

PART C: COMBINED ENGINEERING OBSERVATIONS

Although the experiments provided useful results, several limitations should be considered.

1. **Accelerated failures:** The failure rates used in the simulation were higher than those normally expected in production environments. The resulting downtime values should not be interpreted as actual production service-level agreements.
2. **Simplified network model:** Network communication was represented using fixed round-trip times and simplified message-loss probabilities. Real networks experience additional delays, congestion and jitter.

3. **Simulated workload:** The workload was generated using mathematical distributions rather than actual telecom traffic records.
4. **Byzantine failures:** Byzantine behaviour was analysed theoretically but was not included in the simulation.
5. **Simplified leader election:** Leader election was represented using a fixed delay rather than a complete distributed consensus protocol.
6. **Limited random seeds:** The experiments used eight seeds for Milestone 7 and five seeds for Milestone 8. More experiments would provide stronger statistical confidence.
7. **Parameter sensitivity:** Some parameters, such as migration cost, network latency and workload distribution, were not exhaustively tested.

These limitations mean that the results should be treated as model-based comparisons rather than direct measurements of a production telecom network.

General Conclusion

The practical successfully demonstrated how fault tolerance, recovery mechanisms and distributed computing models influence the performance of distributed systems.

Milestone 7 established that replication significantly improves service availability compared to a single-node architecture. It also demonstrated that quorum-based coordination prevents split-brain exposure, although it introduces an availability trade-off under frequent failures.

Milestone 8 demonstrated that processor pooling and workload migration improve performance when workloads are unevenly distributed. It also showed that integrated architectures can provide good performance when processing capacity is allocated according to the workload.

For the proposed Distributed Telecom Resource Management System, the findings support the use of a capacity-matched integrated architecture combined with appropriate replication and quorum-based fault-tolerance mechanisms.

Overall, these milestones improved my understanding of how distributed systems can be designed to remain available, distribute workloads efficiently and recover from failures.

PART D: REPRODUCIBILITY AND RESEARCH METHOD

Development Environment

The simulations were implemented using Python 3 and the standard library.

The following environment was used:

Component	Specification
Programming language	Python 3.13.18
Operating system	Linux
External libraries	None
Main program	sim_w7_w8.py
Results directory	results/

M7 random seeds	1–8
M8 random seeds	1–5
Random number generator	Python random.Random
Confidence interval	95%

Execution Procedure

The simulation was executed using the following command:

python3 sim_w7_w8.py --out results --seeds 8 --m8-seeds 5

The execution generated the following output files:

- results/results.json – Contains the raw and aggregated experimental results.
- results/m7_tables.md – Contains the Milestone 7 result tables.
- results/m8_tables.md – Contains the Milestone 8 result tables.
- results/run_stdout.txt – Contains the console output from the simulation.

The use of explicit random seeds made the experiments reproducible using the same program and parameters.

Click here for Practical_1 code